Lecture 12

Overloading methods in java

By: Rashida Naz

Agenda

- **Overloading methods**
- Overloading constructors
- Parameters and arguments
- Pass by value
- Pass by reference
- Objects as parameters and arguments

Method overloading

- allows a class to have multiple methods with the same name but different parameters, enabling compile-time polymorphism.
- Methods can share the same name if their parameter lists differ.
- Cannot overload by return type alone; parameters must differ.
- The compiler chooses the most specific match when multiple methods could apply.

The different ways of method overloading in Java

1. Changing the Number of Parameters

- Method overloading can be achieved by changing the number of parameters when passing to different methods.

Example:

```
class Product{

    // Multiplying two integer values
    public int multiply(int a, int b){

        int prod = a * b;
        return prod;
    }

    // Multiplying three integer values
    public int multiply(int a, int b, int c){

        int prod = a * b * c;
        return prod;
    }
}
```

Example:

```
class Demo{
    public static void main(String[] args)
    {
        Product ob = new Product();
        // Calling method to Multiply 2 numbers
        int prod1 = ob.multiply(1, 2);
        // Printing Product of 2 numbers
        System.out.println(
            "Product of the two integer value: " + prod1);
        // Calling method to multiply 3 numbers
        int prod2 = ob.multiply(1, 2, 3);
        // Printing product of 3 numbers
        System.out.println(
            "Product of the three integer value: " + prod2);
    }
}
```

The different ways of method overloading in Java

2. Changing Data Types of Parameters

- In many cases, methods can be considered overloaded if they have the same name but have different parameter types, methods are considered to be overloaded.

Example 2

```
class Product{

    public int prod(int a, int b, int c){
        return a * b * c;

    }
    public double prod(double a, double b, double
c){
        return a * b * c;
    }
}
```

The different ways of method overloading in Java

- **3. Changing the Order of Parameters**
- Method overloading can also be implemented by rearranging the parameters of two or more overloaded methods.

Example 3

```
class Student {

    public void studentId(String name, int rollNo){

        System.out.println("Name: " + name
                           + ", Roll-No: " + rollNo);
    }
    public void studentId(int rollNo, String name){

        System.out.println("Roll-No: " + rollNo
                           + ", Name: " + name);
    }
}
```

What if the Exact Prototype Does Not Match?

- Java tries type promotion in overloading
- Convert to a higher type in the same hierarchy (e.g., byte -> int).
- Convert to the next higher hierarchy if needed (e.g., int -> float).

If no suitable method is found,
it causes a compilation error.

Example 4

```
class Demo{
    public void show(int x){
        System.out.println("In int: " + x);
    }
    public void show(String s){
        System.out.println("In String: " + s);
        }
    public void show(byte b){
        System.out.println("In byte: " + b);
    }
}
```

Example 4

```
public class UseDemo {
    public static void main(String[] args) {
        Demo obj = new Demo();
        obj.show((byte) 25);
        obj.show("hello");
        obj.show(250);
        obj.show('A');
        // obj.show(7.5);      // Error: no suitable method for
double
    }
}
```

Overloading constructors

Agenda

- **Overloading methods**
- **Overloading constructors**
- Parameters and arguments
- Pass by value
- Pass by reference
- Objects as parameters and arguments

Constructor overloading

- in Java is a feature that allows a single class to have multiple constructors, provided each has a unique parameter list (different in number, type, or order of parameters).
- It enables flexible object creation, allowing objects to be initialized in different ways depending on the available data.

Key Concepts and Rules

- **Same Name**: All overloaded constructors must share the same name as the class they belong to.
- **Different Signatures**: The primary requirement is that each constructor's signature (its parameters) must be distinct. The return type cannot be used to differentiate constructors, as they do not have a return type (not even void).
- **Compile-Time Polymorphism**: The Java compiler determines which specific constructor to invoke at compile time based on the arguments provided during object creation. This makes it an example of static polymorphism.

Key Concepts and Rules

- **this() Keyword**: One constructor can call another constructor within the same class using the this() keyword to avoid code duplication (known as constructor chaining). The call to this() must be the first statement in the constructor body.
- **Default Constructor**: If you define any parameterized constructor in a class, the Java compiler will *not* automatically provide a default (no-argument) constructor. If you still need a no-argument constructor, you must explicitly define it.

```
public class Student {
    String name;
    int age;

    // Default constructor (no arguments)
    public Student() {
        this("Unknown", 0); // Calls the two-parameter constructor
    }

    // Constructor with one parameter (name)
    public Student(String name) {
        this(name, 0); // Calls the two-parameter constructor
    }

    // Constructor with two parameters (name and age)
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
```

Code continued

```
public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }

    public static void main(String[] args) {
        Student s1 = new Student();           // Calls the default constructor
        Student s2 = new Student("Alice");    // Calls the one-parameter constructor
        Student s3 = new Student("Bob", 30); // Calls the two-parameter constructor

        s1.display();
        s2.display();
        s3.display();
    }
}
```

Benefits

- Flexibility: Provides multiple ways to initialize an object based on different scenarios and available data.
- Code Reusability: Facilitates the reuse of initialization logic through constructor chaining, reducing redundant code.
- Readability: Allows for clear, descriptive constructors tailored to specific data input needs, making the code easier to understand.
- Dynamic Initialization: Objects can be initialized with partial or complete data at runtime

Agenda

- **Overloading methods**
- **Overloading constructors**
- **Parameters and arguments**
- Pass by value
- Pass by reference
- Objects as parameters and arguments

Parameters and Arguments

Arguments and Parameters are closely related but refer to different concepts in method invocation and definition.

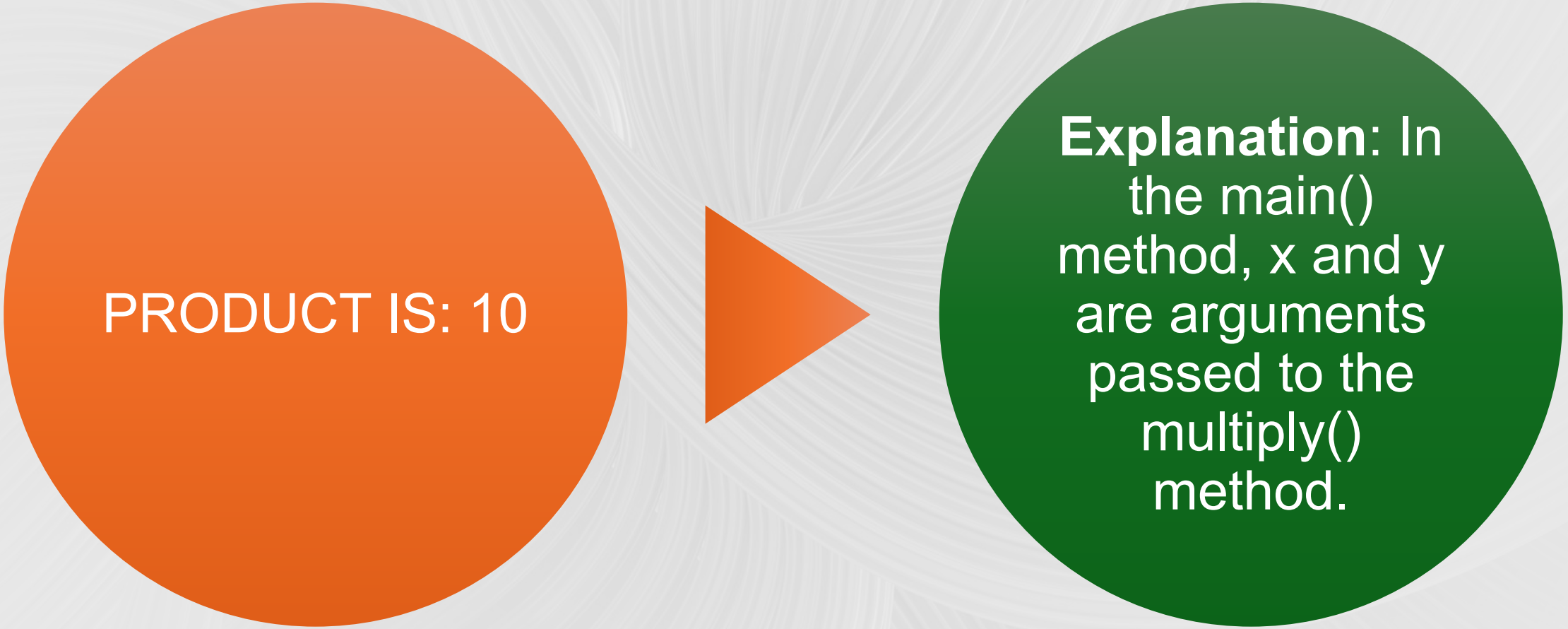
Parameters and Arguments

Argument
An argument is a value passed to a function at the time of its call, which is used by the function during execution.
These values replace the variables defined in the function and allow it to work with actual data.
Arguments are supplied when a function is invoked
They provide input values to the function
Passed arguments replace the parameters defined in the function
The function executes using these passed values

Example

```
public class Example {

 public static int multiply(int a, int b) {
return a * b; }

public static void main(String[] args) {
int x = 2; int y = 5;
int product = multiply(x, y);
System.out.println("PRODUCT IS: " + product);

}

}
```

Output


PRODUCT IS: 10
Explanation: In the main() method, x and y are arguments passed to the multiply() method.

Parameter

- A parameter is a variable defined in a function declaration that represents the data the function will receive. It acts as a placeholder for the values that are passed when the function is called.
- Parameters are defined during function declaration
- They act as variables inside the function
- They receive values from arguments at runtime
- Parameters and arguments may hold the same value but are conceptually different

Example

```
public class Example {

    public static int multiply(int a, int b) {
        return a * b;
    }
    public static void main(String[] args) {
        int x = 2;
        int y = 5;

        int product = multiply(x, y);
        System.out.println("PRODUCT IS: " + product);
    }
}
```

PRODUCT IS: 10

Explanation: In
the multiply() method, a and b are **parameters** that
receive values from the arguments x and y

Difference between an Argument and a Parameter

Argument	Parameter
Values passed to a method during a method call	Variables defined in the method declaration
Used in the calling method	Used in the called method
Send data to the method	Receive data from arguments
Also called Actual Parameters	Also called Formal Parameters
Exist at the time of method invocation	Exist during method execution

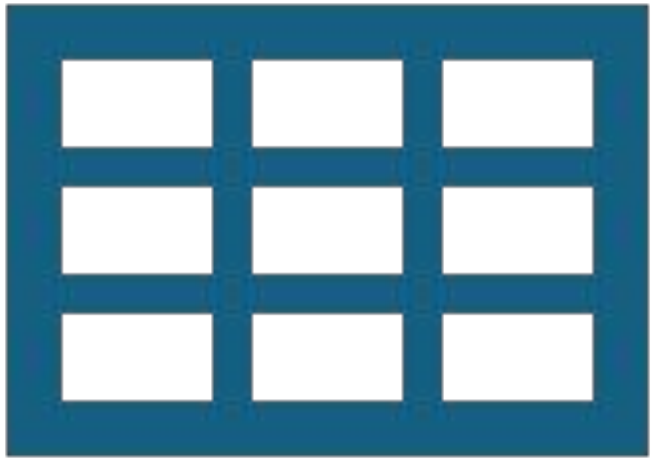
Agenda

- **Overloading methods**
- **Overloading constructors**
- **Parameters and arguments**
- **Pass by value**
- **Pass by reference**
- Objects as parameters and arguments


Arrays
Arrays
Reference
Reference

Pass By Value: Primitive Types

- In Java, **primitive types** such as int, double, and byte are passed to methods by value.
- This means that when a primitive value is passed as a parameter, only a **copy** of the value is passed, and the original value remains unchanged.



Example

```
public class Tipler {
public static void main(String[]
args) {
int a = 5;
System.out.println("Old value: " +
a);
change(a);
System.out.println("New value: " +
a);
}

static void change(int a) {
a = a + 10;
}
}
```

Output:

Old value: 5
New value: 5

Explanation

In this example, the variable a is printed first. Then, it is passed to the change method, where 10 is added to its value.

However, when the method finishes, the original variable a remains unchanged because **only a copy of the value** was modified within the method.

Pass By Reference: Reference Types

- In contrast, **reference types** such as objects and arrays are passed to methods by reference.
- When a reference type is passed as a parameter, the **address** of the object in memory is passed, allowing the method to modify the original object.

Example

```
public class Tipler {
int x;

public static void main(String[] args) {
Tipler t1 = new Tipler();
t1.x = 5;
System.out.println("Old value: " + t1.x);
t1.change(t1);
System.out.println("New value: " + t1.x);
}

void change(Tipler t1) {
t1.x = t1.x + 10;
}
}
```

Output:
Old value: 5
New value: 15

Explanation:

- In this example, the t1 object's x property is initially set to 5. When the object is passed to the change method, its x value is increased by 10. Since the **reference** to the original object was passed, the modification is reflected in the original object.

Arrays as Reference Types

- Arrays are also treated as reference types in Java. When an array is passed as a parameter, changes to its elements affect the original array.

Example

```
public class Tipler {
public static void main(String[] args) {
int[] array = {1, 6, 9};
System.out.println("Old value: " + array[0]);
change(array);
System.out.println("New value: " + array[0]);
}

static void change(int[] array) {
array[0] = array[0] + 5;
}
}
```

Output:

- Old value: 1
  New value: 6

Explanation:

- In this example, the first element of the array is modified within the change method. The modification affects the original array because arrays are reference types.

Combining Primitive and Reference Types: what will be the output of the following code?

```
public class Tipler {
int b;

public static void main(String[] args) {
int a = 4;
change1(a);
System.out.println(a);

Tipler d1 = new Tipler();
d1.b = 10;
d1.change(d1);
System.out.println(d1.b);
}

static void change1(int a) {
a = a + 25;
}

void change(Tipler d1) {
d1.b = d1.b + 25;
}
}
```

Agenda

- **Overloading methods**
- **Overloading constructors**
- **Parameters and arguments**
- **Pass by value**
- **Pass by reference**
- **Objects as parameters and arguments**

Objects as parameters and arguments

- In Java, objects can be passed to methods just like primitive data types. This is a core feature of object-oriented programming that allows methods to access and manipulate an object's data.

- Java is strictly **pass-by-value**. However, for objects, the "value" being passed is the **reference** (memory address) to the actual object, not the object itself. This leads to an effect similar to pass-by-reference.

Objects as parameters and arguments

- To be continued

References

- https://medium.com/@asli.beyhann/pass-by-value-and-pass-by-reference-in-java-an-in-depth-exploration-9c0e4534721e